

Bootcamp: Full Stack Junior

Modulo 3: Backend con PHP y Laravel

Semana 8

Contenido

1

Eloquent ORM y Query Builder (consultas eficientes).

2

Estándares RESTful y gestión de errores HTTP.

3

Arquitectura Limpia en Laravel.

4

Configuración de CORS.

Contenido:

1. Eloquent ORM y Query Builder (consultas eficientes).

Eloquent ORM y Query Builder son herramientas clave de Laravel para interactuar con bases de datos.

1.1. Eloquent ORM

Eloquent es un ORM (Object-Relational Mapping) que mapea tablas de base de datos a clases PHP (modelos), permitiendo operaciones como crear, leer, actualizar y eliminar (CRUD) de forma intuitiva y orientada a objetos. Cada modelo representa una tabla; por convención, usa el plural snake_case del nombre de la clase (ej: Flight → flights). Incluye relaciones (uno a muchos, polimórficas), timestamps automáticos (created_at, updated_at) y soft deletes.

Ejemplo básico en un modelo:



```
php artisan make:model User
```

Ilustración 1 - Generación de modelos - Fuente: Autor de contenidos



```
class User extends Model {  
    protected $fillable = ['name', 'email'];  
}  
$users = User::all(); // Colección de modelos  
$user = User::create(['name' => 'Juan', 'email' => 'juan@email.com']);
```

Ilustración 2 - Modelo y colección de modelo - Fuente: Autor de contenidos

1.2. Query Builder

Query Builder es una interfaz fluida para construir consultas SQL sin escribir SQL crudo, usando el facade DB y compatible con todos los motores de BD de Laravel. Retorna colecciones de objetos stdClass, no modelos Eloquent, y es ideal para consultas complejas o raw. Soporta joins, wheres avanzados, agregados (count, sum), chunking para grandes datasets y protecciones contra SQL injection vía bindings.

Ejemplo básico:

```
$users = DB::table('users')
->where('votes', '>', 100)
->orderBy('name')
->get();
```

Ilustración 3 - Query Builder - Fuente: Autor de contenidos

1.3. Diferencias clave

Aspecto	Eloquent ORM	Query Builder
Enfoque	Orientado a objetos (modelos)	Fluido, SQL-like (DB::table)
Retorno	Instancias de modelos	Objetos stdClass
Relaciones	Nativo (hasMany, belongsTo)	Manual (joins)
Rendimiento	Similar; Eloquent delega a QB	Más directo para queries raw
Uso ideal	Apps con lógica de negocio	Consultas complejas/masivas

Tabla 1 - Eloquent VS Query Builder - Fuente: Autor de Contenidos

2. Estándares RESTful y gestión de errores HTTP.

Los estándares RESTful en Laravel 12 promueven el uso de rutas de recursos y respuestas JSON consistentes para APIs stateless. La gestión de errores HTTP se centraliza en el handler de excepciones, permitiendo respuestas estandarizadas como 404 o 422 para cumplir con convenciones REST.

Habilitar el soporte de API

A partir de Laravel 11 y continuando en la versión 12, las rutas de API no vienen habilitadas por defecto para mantener el proyecto limpio. Debes ejecutar el siguiente comando:

```
php artisan install:api
```

Ilustración 4 - Manejo de errores - Fuente: Autor de contenidos

¿Qué hace este comando?

- Crea el archivo routes/api.php.
- Instala **Laravel Sanctum** para la autenticación por tokens.

- Configura la base de datos para manejar tokens personales.

2.1. Rutas RESTful

Laravel 12 facilita APIs RESTful mediante `Route::apiResource` en `routes/api.php`, que genera rutas CRUD automáticas con prefijo `/api` y sin métodos `create/edit` (solo `index`, `show`, `store`, `update`, `destroy`).

- Usa verbos HTTP correctos: GET (leer), POST (crear), PUT/PATCH (actualizar), DELETE (eliminar).
- Ejemplo: `Route::apiResource('users', UserController::class);` crea `/api/users` (GET/POST), `/api/users/{user}` (GET/PUT/DELETE).
- Para colecciones, usa `UserResource::collection($users)` o paginación con `User::paginate()->toResourceCollection()`.

2.2. Controladores

Genera controladores con `php artisan make:controller UserController --api --model=User`, que incluye métodos esqueléticos para acciones REST.

Los controladores inyectan modelos vía `route model binding`: `public function show(User $user) { return new UserResource($user); }`.

Retorna recursos JSON: `return new UserResource($user);` para transformar datos de forma consistente, incluyendo relaciones condicionales con `whenLoaded('posts')`.

2.3. Respuestas JSON

Usa clases de recursos Eloquent (`php artisan make:resource UserResource`) para serializar modelos a JSON, con `toArray()` personalizable y `wrapping` automático en `data`.

- Colecciones: `UserResource::collection(User::all());`.
- Paginación incluye links y meta automáticamente.
- Desactiva `wrapping global`: `JsonResource::withoutWrapping();` en `AppServiceProvider`.

2.4. Gestion de Errores HTTP

Gestión de Errores HTTP en Laravel



Centralización de Errores

Configura en `bootstrap/app.php`

`withExceptions();`

Report y Render Centralizados

Reporte & Renderizado

`report()`

Reporte de Errores



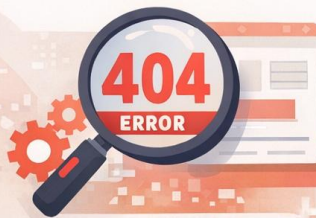
`render()`

Respuestas JSON

Manejo de Respuestas de API

```
$exceptions->render(function (NotFoundException $e, Request $request) {  
    if ($request->expectsJson()) {  
        return response()->json(["message" => "Not Found"], 404);  
    }  
});
```

⚠ Devuelve JSON personalizado si no se encuentra la ruta.



Respuesta JSON:

`{"message": "Not Found"}`

Manejo Eficiente de Errores en Laravel

Ilustración 5 - Manejo de errores - Fuente: GPT Image

3. Arquitectura Limpia en Laravel

La Arquitectura Limpia busca desacoplar la **lógica de negocio** (lo que hace tu empresa) de los **detalles de implementación** (la base de datos, el framework, las APIs externas).

- **Separación de responsabilidades:** Cada clase tiene una única razón para cambiar. Si decides cambiar el motor de base de datos, no deberías tocar la lógica de cómo se calcula un descuento.
- **Independencia del framework:** Laravel se convierte en un "detalle". Si en el futuro quisieras usar otro motor o una consola de comandos, el núcleo de tu aplicación debería seguir funcionando.
- **Mantenibilidad y escalabilidad:** Al tener fronteras claras, es más fácil testear componentes por separado y agregar nuevas funcionalidades sin romper las existentes.

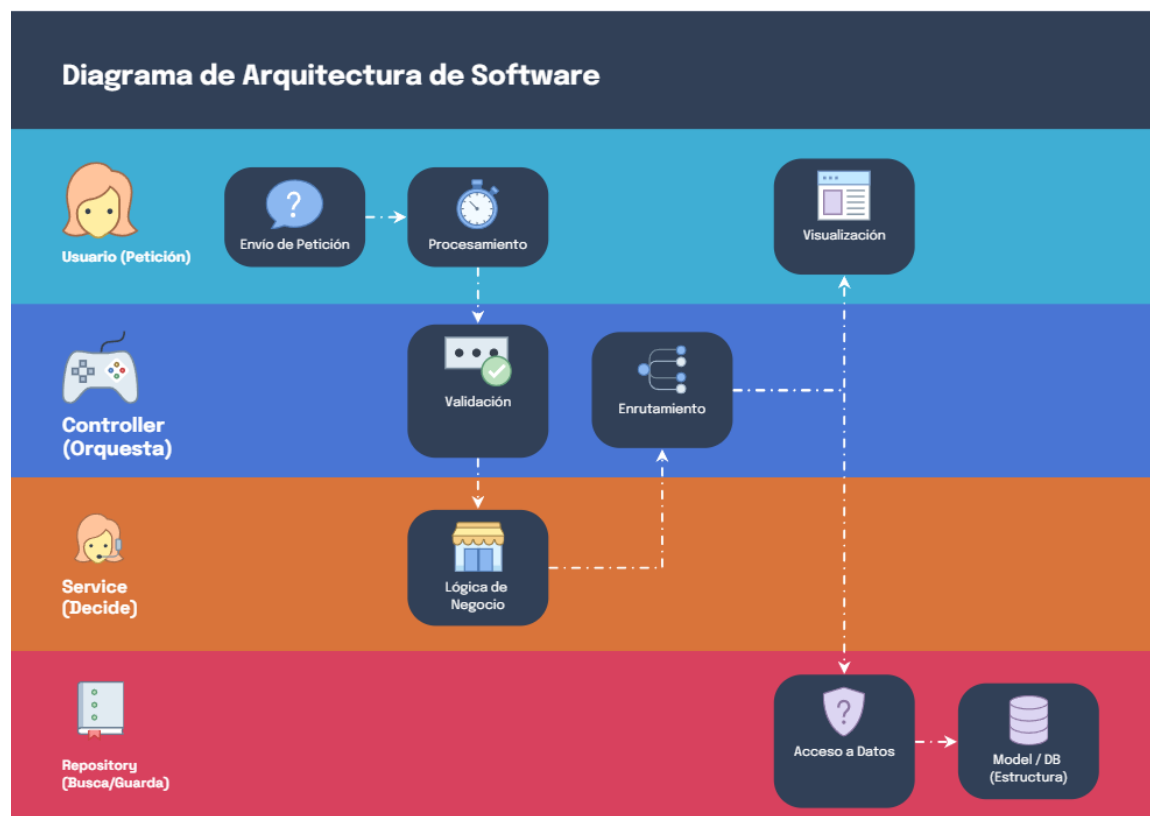


Ilustración 6 - Arquitectura de Software - Fuente: Autor de contenidos

3.1. Capas de Laravel

Para aplicar esto en Laravel, dividimos el código en capas específicas:

1. **Requests:** Se encargan exclusivamente de la **validación** y autorización de los datos de entrada.
2. **Controllers:** Son simples "pasamanos". Reciben la petición, llaman al Service correspondiente y devuelven una respuesta (JSON o Vista). **No deben contener lógica de negocio.**
3. **Services:** Aquí reside el "corazón" de la aplicación. Cálculos, integraciones con APIs y reglas de negocio.
4. **Repositories:** Actúan como una capa de abstracción sobre Eloquent. Si necesitas cambiar una consulta SQL compleja o usar caché, lo haces aquí.
5. **Models:** Representan la estructura de los datos y sus relaciones. En Arquitectura Limpia, se mantienen "delgados".

Capa	Función Principal	Lo que NO debe hacer
Controller	Orquestar el flujo y retornar respuestas.	Consultas a la base de datos o cálculos.
Service	Ejecutar la lógica de negocio y reglas.	Leer directamente Input::get() o manejar HTTP.
Repository	Abstracción del acceso a datos (CRUD).	Decidir si un usuario tiene permiso para algo.
Model	Definir estructura, relaciones y casts.	Contener lógica compleja de la aplicación.

Tabla 2 - Capas de Laravel - Fuente: Autor de Contenidos

4. Configuración de CORS.

4.1. ¿Qué es CORS?

CORS (*Cross-Origin Resource Sharing*) es un mecanismo de seguridad basado en HTTP que permite a un servidor indicar cualquier **origen** (dominio, esquema o puerto) distinto al suyo desde el cual un navegador debería permitir la carga de recursos.

- **Origen cruzado:** Ocurre cuando, por ejemplo, tu frontend en <http://localhost:3000> intenta consultar tu API en <http://api.misitio.com>. El navegador bloquea esto por defecto para prevenir ataques.
- **Problemas comunes:** El error más frecuente es el famoso No 'Access-Control-Allow-Origin' header is present, que sucede cuando el servidor no devuelve las cabeceras adecuadas durante la petición de "preflight" (solicitud OPTIONS).
- **Seguridad:** CORS no es una herramienta para prevenir ataques de hackers, sino una protección para los usuarios finales, evitando que sitios maliciosos realicen peticiones en su nombre hacia otros dominios donde tengan sesión iniciada.

4.2. Configuración de CORS en Laravel

Desde Laravel 7 en adelante, el soporte para CORS viene integrado de forma nativa a través del paquete `fruitcake/laravel-cors` (ahora parte del core).

Archivo `config/cors.php`

Este archivo centraliza toda la configuración. Los puntos clave son:

- **Paths:** Define qué rutas de tu aplicación aceptarán peticiones de otros dominios (ej. `api/*`).
- **Allowed Origins:** Aquí defines los dominios específicos (ej. <https://mi-app.com>). Usar `*` permite cualquier origen, lo cual es útil en desarrollo, pero arriesgado en producción.
- **Allowed Methods:** Define si permites GET, POST, PUT, DELETE, etc.

- **Allowed Headers:** Especifica qué cabeceras personalizadas puede enviar el cliente (ej. Content-Type, X-Requested-With, Authorization).

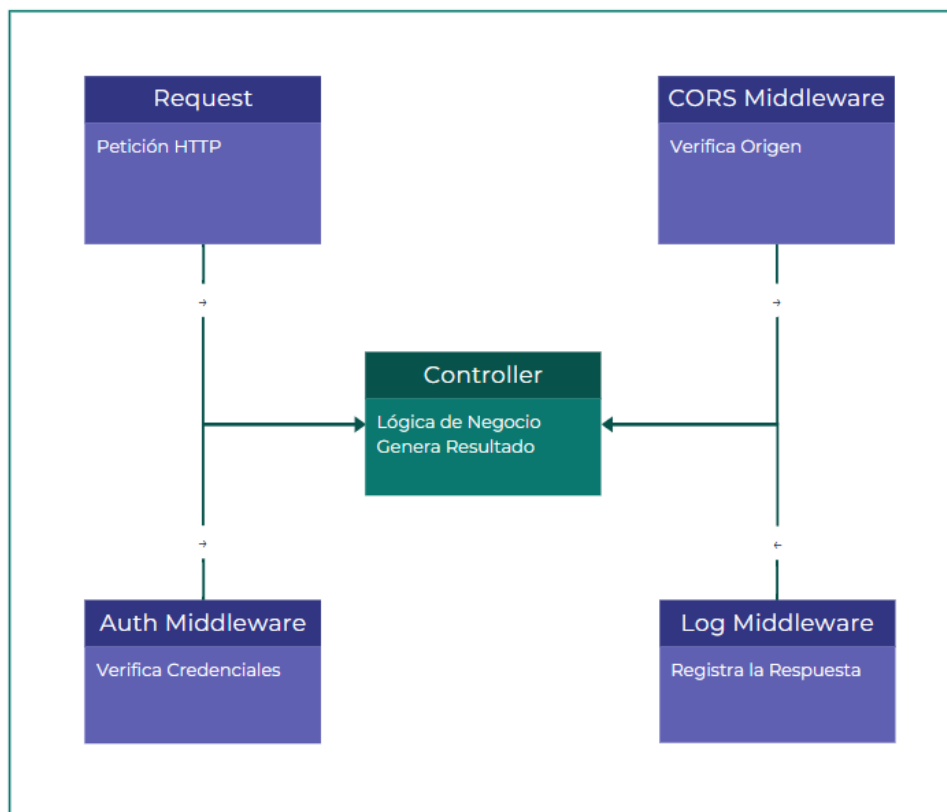
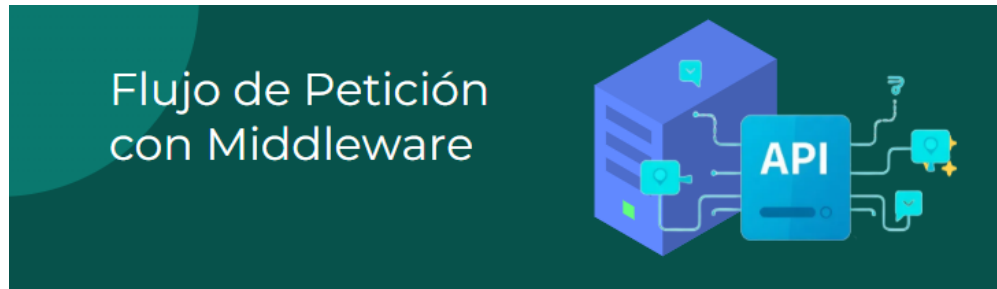


Ilustración 7 - Flujo de Middleware - Fuente: Autor de contenidos

Recursos de Apoyo



Video

[Instalacion de funcionalidad de API Rest en Laravel 12.](#)



Video

[REST API CRUD en Laravel.](#)



Material de Lectura

[Recurso oficial de Laravel para transformar Eloquent models en respuestas JSON eficientes en APIs REST.](#)